PRAKTIKUM KECERDASAN KOMPUTASIONAL

Menyusun Rute Kunjungan dengan Algoritma Genetika

Lembar soal dan kerangka program Python

1 Pendahuluan Soal

Perpustakaan kota mengoperasikan mobil layanan keliling untuk mengantarkan paket buku ke titik-titik baca di beberapa wilayah. Selama ini, urutan kunjungan ditentukan berdasarkan kebiasaan pengemudi. Ketika jumlah titik bertambah, rute yang tampak masuk akal belum tentu menghasilkan perjalanan paling singkat.

Untuk jadwal esok hari, mobil berangkat dari **depo pusat**, mengunjungi **delapan titik layanan** A–H, lalu kembali ke depo. Setiap titik harus dikunjungi **tepat satu kali**. Seluruh paket dianggap sudah berada di mobil sejak awal, sehingga kapasitas kendaraan dan waktu pelayanan tidak menjadi batasan pada latihan ini. Keputusan yang perlu dibuat adalah **urutan kunjungan** yang meminimalkan jarak tempuh total.

Peta kerja disederhanakan menjadi koordinat dua dimensi dalam kilometer. Jarak antarlokasi dihitung sebagai garis lurus dengan rumus

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

Rumus ini merupakan model latihan; kondisi jalan dan lalu lintas tidak dimasukkan. Koordinat yang sama juga akan dipakai untuk menggambar rute hasil pencarian.

Lokasi	x	y	Lokasi	x	y
Depo	0	0	D	11	3
A	2	5	E	8	0
B	5	8	F	5	2
C	9	7	G	2	-2
			H	-1	3

Titik berangkat dan titik akhir selalu depo. Dengan delapan titik layanan, terdapat $8! = 40\,320$ urutan kunjungan yang mungkin sebelum mempertimbangkan bahwa sebagian rute memiliki jarak yang sama jika dibalik. Gunakan algoritma genetika untuk mencari rute pendek, kemudian tampilkan urutannya pada peta.

2 Ketentuan Pengerjaan

Lengkapi setiap bagian bertanda `TODO` pada Blok 1–6. Salin seluruh blok kode **sesuai urutan** ke dalam satu berkas Python, misalnya `soal_rute.py`, termasuk fungsi visualisasi pada Blok 7. Jalankan berkas tersebut setelah semua fungsi selesai; Blok 8 akan memeriksa hasil dan membuat gambar. Baris `raise NotImplementedError(...)` menandai tempat yang harus diganti dengan implementasi Anda.

1. Gunakan **permutasi** angka 1–8 sebagai kromosom. Angka 0 menyatakan depo, sedangkan 1–8 menyatakan A–H. Contoh awal kromosom `[1, 3, 2, ...]` berarti mengunjungi A, C, B, lalu titik-titik berikutnya.

2. Jumlah anggota populasi tetap pada setiap generasi. Gunakan seleksi turnamen, crossover yang menjaga setiap titik muncul sekali, mutasi dengan pertukaran dua posisi, dan elitisme.

3. Nilai fitness harus **lebih tinggi** untuk rute yang lebih pendek. Rute terbaik yang dilaporkan harus tetap valid dan mencantumkan jarak totalnya.

4. Pertahankan antarmuka fungsi pada kerangka kode agar seluruh blok dapat dijalankan bersama. Anda boleh menambah fungsi pembantu jika diperlukan. Gunakan `matplotlib` untuk melihat peta rute; pasang dengan `python3 -m pip install matplotlib` jika belum tersedia.

3 Blok Kode yang Harus Dilengkapi

3.1 Blok 1 — Data dan populasi awal

Data berikut sudah disediakan. Lengkapi `buat_individu` agar menghasilkan urutan acak titik 1–8, tanpa titik yang berulang. Fungsi `jarak_antar` menghitung jarak dari koordinat tanpa membulatkan hasilnya.

```
import math
import random

NAMA = ("Depo", "A", "B", "C", "D",
        "E", "F", "G", "H")
POSISI = (
    (0, 0), (2, 5), (5, 8), (9, 7), (11, 3),
    (8, 0), (5, 2), (2, -2), (-1, 3),
)

def jarak_antar(i, j):
    x1, y1 = POSISI[i]
    x2, y2 = POSISI[j]
    return math.hypot(x2 - x1, y2 - y1)

def buat_individu(rng):
    rute = list(range(1, len(NAMA)))
    # TODO: acak rute menggunakan rng
    raise NotImplementedError("Blok 1: acak rute")
    return rute
```

3.2 Blok 2 — Jarak rute dan fitness

Lengkapi perhitungan jarak mulai dari depo, melewati seluruh titik pada `rute`, lalu kembali ke depo. Setelah itu, buat fitness yang selalu lebih tinggi untuk jarak yang lebih pendek. Salah satu pilihan ialah $F(r) = 1/(1 + d(r))$, dengan $d(r)$ jarak total.

```
def jarak_total(rute):
    # TODO: jumlahkan jarak Depo -> titik pertama,
    #       antar titik berurutan, lalu titik terakhir -> Depo
    raise NotImplementedError("Blok 2: jarak total")

def fitness(rute):
    # TODO: kembalikan 1 / (1 + jarak_total(rute))
    raise NotImplementedError("Blok 2: fitness")
```

3.3 Blok 3 — Seleksi induk

Dalam satu turnamen, ambil `ukuran` individu berbeda secara acak dari populasi dan pilih yang memiliki fitness tertinggi. Kembalikan **salinan** pemenang agar perubahan pada anak tidak mengubah populasi lama.

```
def turnamen(populasi, rng, ukuran=3):
    peserta = rng.sample(populasi, ukuran)
    # TODO: pilih peserta dengan fitness terbesar
    #       dan kembalikan salinan list-nya
    raise NotImplementedError("Blok 3: seleksi turnamen")
```

3.4 Blok 4 — Crossover untuk urutan kunjungan

Crossover tidak boleh menghasilkan titik ganda atau menghilangkan titik. Gunakan cara berikut: pilih dua batas `awal` dan `akhir`, salin bagian `induk1[awal:akhir]` ke anak, lalu isi posisi anak yang kosong dengan titik dari `induk2` menurut urutan kemunculannya.

Contoh dengan `awal=2` dan `akhir=5`:

Induk 1:	[1, 2, 3, 4, 5, 6, 7, 8]
Induk 2:	[4, 5, 8, 2, 1, 7, 6, 3]
Segmen yang disalin:	[3, 4, 5]
Anak setelah dilengkapi:	[8, 2, 3, 4, 5, 1, 7, 6]

```
def crossover_urutan(induk1, induk2, rng):
    n = len(induk1)
    awal, akhir = sorted(rng.sample(range(n), 2))
    akhir += 1  # batas akhir slice tidak ikut disalin
    anak = [None] * n
    # TODO: salin segmen induk1[awal:akhir] ke anak
    # TODO: buat daftar titik dari induk2 yang belum ada di anak
    # TODO: isi posisi None dari kiri ke kanan memakai daftar itu
    raise NotImplementedError("Blok 4: crossover urutan")
    return anak
```

3.5 Blok 5 — Mutasi pertukaran

Dengan peluang `peluang`, pilih dua posisi berbeda dan tukar isinya. Jika mutasi tidak terjadi, kembalikan salinan rute semula. Pertukaran menjaga setiap titik tetap muncul tepat sekali.

```
def mutasi_tukar(rute, rng, peluang):
    hasil = rute[:]
    # TODO: jika rng.random() < peluang, pilih dua indeks
    #       berbeda dan tukar kedua titik pada hasil
    raise NotImplementedError("Blok 5: mutasi pertukaran")
    return hasil
```

3.6 Blok 6 — Siklus algoritma genetika

Gabungkan fungsi sebelumnya. Pada setiap generasi, salin `elit` individu terbaik, lalu isi tempat yang tersisa dengan anak hasil seleksi, crossover, dan mutasi. Untuk tiap pasangan induk, lakukan crossover dengan peluang `peluang_crossover`; jika tidak, salin induknya. Simpan jarak terbaik sepanjang proses dalam `riwayat`.

```
def jalankan_ga(seed=7, ukuran_populasi=60, generasi=150,
                peluang_crossover=0.85, peluang_mutasi=0.2,
                elit=2):
    rng = random.Random(seed)
    populasi = [buat_individu(rng)
                for _ in range(ukuran_populasi)]
    terbaik = min(populasi, key=jarak_total)[:]
    riwayat = [jarak_total(terbaik)]

    for _ in range(generasi):
        baru = []
        # TODO: salin elit individu dengan jarak terpendek
        #       dari populasi ke baru
        raise NotImplementedError("Blok 6: elitisme")

        while len(baru) < ukuran_populasi:
            induk1 = turnamen(populasi, rng)
            induk2 = turnamen(populasi, rng)
            # TODO: lakukan crossover dengan peluang yang
            #       ditentukan; jika gagal, salin kedua induk
            # TODO: mutasikan kedua anak
            raise NotImplementedError("Blok 6: anak baru")
            for anak in (anak1, anak2):
                if len(baru) < ukuran_populasi:
                    baru.append(anak)

        populasi = baru
        kandidat = min(populasi, key=jarak_total)
        if jarak_total(kandidat) < jarak_total(terbaik):
            terbaik = kandidat[:]
        riwayat.append(jarak_total(terbaik))

    return terbaik, riwayat
```

3.7 Blok 7 — Visualisasi rute dan perkembangan GA

Blok berikut **sudah lengkap**. Gambar kiri memperlihatkan lokasi, urutan kunjungan, dan arah perjalanan dari depo kembali ke depo. Gambar kanan menunjukkan perubahan jarak terbaik sepanjang generasi. Fungsi ini menyimpan hasil ke `hasil_rute.png` dan menampilkannya di layar.

```
import matplotlib.pyplot as plt

def gambar_hasil(rute, riwayat):
    lengkap = [0] + rute + [0]
    xs = [POSISI[i][0] for i in lengkap]
    ys = [POSISI[i][1] for i in lengkap]
    fig, (ax_rute, ax_riwayat) = plt.subplots(
        1, 2, figsize=(12, 5))

    ax_rute.plot(xs, ys, "-", color="#006B74",
                 linewidth=1.5, alpha=0.65)
    for i, (x, y) in enumerate(POSISI):
        warna = "#B23A48" if i == 0 else "#15324E"
        ax_rute.scatter(x, y, s=75, color=warna, zorder=3)
        ax_rute.annotate(NAMA[i], (x, y), xytext=(5, 6),
                         textcoords="offset points")
    for asal, tujuan in zip(lengkap, lengkap[1:]):
        ax_rute.annotate(
            "", xy=POSISI[tujuan], xytext=POSISI[asal],
            arrowprops=dict(arrowstyle="->", color="#006B74",
                            lw=1.4, shrinkA=14, shrinkB=14))
    ax_rute.set_title("Rute terbaik: "
                      f"{jarak_total(rute):.2f} km")
    ax_rute.set_xlabel("Koordinat x (km)")
    ax_rute.set_ylabel("Koordinat y (km)")
    ax_rute.set_aspect("equal", adjustable="box")
    ax_rute.grid(alpha=0.3)

    ax_riwayat.plot(range(len(riwayat)), riwayat,
                    color="#006B74", linewidth=2)
    ax_riwayat.set_title("Perkembangan jarak terbaik")
    ax_riwayat.set_xlabel("Generasi")
    ax_riwayat.set_ylabel("Jarak (km)")
    ax_riwayat.grid(alpha=0.3)

    fig.tight_layout()
    fig.savefig("hasil_rute.png", dpi=150)
    plt.show()
```

3.8 Blok 8 — Pemeriksaan dan keluaran

Blok ini juga sudah lengkap. Jalankan setelah Blok 1–7 selesai. Pemeriksaan menguji jarak rute contoh, keabsahan kromosom, dan apakah jarak terbaik yang tercatat tidak memburuk. Gambar akan muncul setelah hasil teks dicetak.

```
if __name__ == "__main__":
    contoh = list(range(1, len(NAMA)))
    assert round(jarak_total(contoh), 2) == 40.06
    assert fitness(contoh) > 0

    rng_uji = random.Random(11)
    p1 = [1, 2, 3, 4, 5, 6, 7, 8]
    p2 = [4, 5, 8, 2, 1, 7, 6, 3]
    anak_uji = crossover_urutan(p1, p2, rng_uji)
    assert sorted(anak_uji) == list(range(1, len(NAMA)))
    hasil_mutasi = mutasi_tukar(p1, rng_uji, 1.0)
    assert sorted(hasil_mutasi) == list(range(1, len(NAMA)))

    rute, riwayat = jalankan_ga()
    assert sorted(rute) == list(range(1, len(NAMA)))
    assert math.isclose(jarak_total(rute), riwayat[-1])
    assert all(a >= b for a, b in zip(riwayat, riwayat[1:]))

    lengkap = [0] + rute + [0]
    print("Rute:", " -> ".join(NAMA[i] for i in lengkap))
    print(f"Jarak total: {jarak_total(rute):.2f} km")
    print(f"Jarak terbaik awal/akhir: "
          f"{riwayat[0]:.2f} / {riwayat[-1]:.2f} km")
    gambar_hasil(rute, riwayat)
```

Kumpulkan: program Python lengkap dan gambar `hasil_rute.png`.